Crayon

Mary love painting so much, but as we know she can't draw very well.
There is no one appreciate her works, so she come up with a puzzle with herself...

Description

There are only one case in each input file, the first line is a integer N (N <= 1,000,00) denoted the total operations executed by Mary.
Then following N lines, each line is one of the folling operations.

- D L R: draw a segment [L, R], 1 <= L <= R <= 1,000,000,000

- C I: clear the ith added segment.
  It's guaranteed that the every added segment will be cleared only once.

- Q L R: query the number of segment sharing at least a common point with interval [L, R]. 1 <= L <= R <= 1,000,000,000. Input n (then following n operations ...)

Output

(for each query, print the result on a single line ...)

Example

Input:
6
D 1 3
D 2 4
Q 2 3
D 2 4
C 2
Q 2 3

Output:
2
2

You have operations:

- D L R → add a segment [L,R]
- C i → delete the i-th added segment (each deleted once)
- Q L R → count how many currently active segments overlap with [L,R]

Two segments overlap if:

max(L1, L2) ≤ min(R1, R2)

Constraints:

- Up to 1e6 operations
- Coordinates up to 1e9
- Dynamic add & delete

Turn the Query Into a Counting Problem

Let's define:

- All active segments
- Count those where:

  start ≤ R
  AND
  end ≥ L

Instead of counting overlaps directly, count:

(total segments with start ≤ R)
-
(segments with start ≤ R AND end < L)

This is classic inclusion–exclusion thinking.

We separate the problem into two independent dimensions:

Dimension 1: segment starts
Dimension 2: segment ends

We track:

- how many segments start at each coordinate
- how many segments end at each coordinate

And we count cleverly.

What Fenwick Trees Are Used For

We use two Fenwick Trees (both over compressed coordinates):

BIT_1 — counts segment starts
BIT_start[x] = number of active segments starting at x

BIT_2 — counts segment ends
BIT_end[x] = number of active segments ending at x

How Each Operation Works

D L R (add segment)

Let cl = compressed(L), cr = compressed(R).

BIT_start.update(cl, +1);
BIT_end.update(cr, +1);

Also store (L, R) so we can delete later.

C i (delete segment)

We look up the i-th added segment (L, R):

BIT_start.update(cl, -1);
BIT_end.update(cr, -1);

Q L R (count overlaps)

We want:

segments with start ≤ R
-
segments with end < L

Fenwick form:

answer =
BIT_start.query(comp(R))
-
BIT_end.query(comp(L) - 1)

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;

    Fenwick(int n) : n(n), bit(n + 1, 0) {}

    void update(int i, int v) {
        for (; i <= n; i += i & -i)
            bit[i] += v;
    }

    int query(int i) const {
        int s = 0;
        for (; i > 0; i -= i & -i)
            s += bit[i];
        return s;
    }

    int rangeQuery(int l, int r) const {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N;
    cin >> N;

    vector<tuple<char, long long, long long>> ops;
    ops.reserve(N);

    vector<long long> coords;   // for coordinate compression

    for (int i = 0; i < N; i++) {
        char type;
        cin >> type;
        if (type == 'D') {
            long long L, R;
            cin >> L >> R;
            ops.emplace_back(type, L, R);
            coords.push_back(L);
            coords.push_back(R);
        } else if (type == 'Q') {
            long long L, R;
            cin >> L >> R;
            ops.emplace_back(type, L, R);
            coords.push_back(L);
            coords.push_back(R);
        } else { // C
            long long idx;
            cin >> idx;
            ops.emplace_back(type, idx, 0);
        }
    }

    // Coordinate compression
    sort(coords.begin(), coords.end());
    coords.erase(unique(coords.begin(), coords.end()), coords.end());

    auto getId = (long long x) {
        return int(lower_bound(coords.begin(), coords.end(), x) - coords.begin()) + 1;
    };

    int M = coords.size();
    Fenwick bitStart(M), bitEnd(M);

    // store added segments so we can delete by index
    vector<pair<int,int>> added;  // (compressed L, compressed R)
    added.reserve(N + 1);

    int addCount = 0;

    for (auto &[type, x, y] : ops) {
        if (type == 'D') {
            int L = getId(x);
            int R = getId(y);

            bitStart.update(L, +1);
            bitEnd.update(R, +1);

            added.push_back({L, R});
            addCount++;
        }
        else if (type == 'C') {
            int idx = int(x) - 1; // 0-based
            auto [L, R] = added[idx];

            bitStart.update(L, -1);
            bitEnd.update(R, -1);
        }
        else { // Q
            int L = getId(x);
            int R = getId(y);

            int startsBeforeR = bitStart.query(R);
            int endsBeforeL  = bitEnd.query(L - 1);

            cout << startsBeforeR - endsBeforeL << "\n";
        }
    }

    return 0;
}
```